SOFTWARE REQUIREMENTS SPECIFICATION

Picturehouse Jaipur Software Requirements Specification

Functional and quality requirements for the first Jaipur release

Jaipur, Rajasthan | Version 1.0 | 18 October 2026

1. Purpose and system boundary

This specification defines the Jaipur movie-ticket and carpool platform across its React web client, Java Spring Boot API, relational database and external providers. The browser presents catalog, show, seat, checkout, ticket, owner and ride experiences. The API owns identity, authorization, inventory, booking state, price calculations, QR validation and integration workflows. Payment, SMS/OTP, email and WhatsApp providers remain external systems.

2. System context

Component	Responsibility
React frontend	Responsive web UI, accessibility, theme preference, seat map, live updates and customer/owner workspaces.
Java backend	REST APIs and WebSocket events; domain rules; auth/roles; transactional booking; provider adapters.
PostgreSQL	Durable accounts, theatre catalogue, screen/show inventory, holds, bookings, payments, rides, memberships and audit data.
External services	OTP delivery, payment checkout and signed webhooks, email delivery, WhatsApp Business notifications and QR scan device/app.

3. Functional requirements

ID	Requirement
FR-01	The system shall show Jaipur as the initial city and filter theatres and rides by area, date and showtime.
FR-02	A visitor attempting a protected action shall be offered sign-in or signup before continuing.
FR-03	An account shall store unique email and E.164 mobile number and track verification of both channels.
FR-04	The system shall provide customer, theatre-owner and administrator roles; owner writes shall be limited to mapped theatres.
FR-05	Customers shall browse films, certifications, languages, venues, dates, formats and showtimes.
FR-06	For each show, the system shall return seat states and category prices: front ₹250, middle ₹300, recliner ₹500 as configurable INR price values.

ID	Requirement
FR-07	Seat selection shall create an expiring hold. Competing holds or bookings for a seat/show combination shall be rejected atomically.
FR-08	Seat-state changes shall be published only after transaction commit and clients shall refresh inventory after reconnect.
FR-09	The system shall create provider checkout requests with amount, INR currency and idempotency reference; confirmation shall follow a verified webhook.
FR-10	A confirmed booking shall have a unique reference and privacy-safe, signed/opaque QR credential with show and entry-window validation.
FR-11	The system shall queue email ticket/QR delivery and approved WhatsApp template delivery; delivery failures shall be retryable and visible operationally.
FR-12	Authorized staff shall scan/validate an entry QR once and record the validation result and time.
FR-13	Theatre owners shall create/update venue, screen, movie and showtime details and view aggregate occupancy.
FR-14	Customers shall create ride posts linked to a show with pickup, dropoff, departure, open seats and per-person contribution.
FR-15	Ride participants shall request to join, accept/decline requests and chat after authentication; contact details shall be access-controlled.
FR-16	The system shall support a light/dark theme and preserve user theme preference.
FR-17	Booking, payment, ticket, seat-hold and owner-write state transitions shall be auditable and idempotent where retried.

4. Data model and ER diagram

The following logical entities show the principal relationships. The detailed schema and normalization notes are in the Project Report; implementation migrations should use PostgreSQL constraints as the final authority for concurrency and integrity.

Core path: ACCOUNT → THEATRE_MEMBER → THEATRE → SCREEN → SHOWTIME ← MOVIE. Customers own BOOKINGS with BOOKING_SEAT and PAYMENT_ATTEMPT records; confirmed bookings have one TICKET_QR. SHOWTIME anchors SEAT_HOLD and RIDE records. RIDE_PARTICIPANT connects accounts to rides, while RIDE_MESSAGE stores ride chat. See the full ER diagram in the Project Report.

5. External interface requirements

Interface	Contract
REST API	Versioned `/api/v1`; JSON; validation errors include stable error code, safe message and correlation ID.
Seat updates	WebSocket/STOMP endpoint; topic scoped to a show; events include show id, seat id, state and event version, never personal data.
Payment	Provider checkout creation and signed webhook; verify signature, amount, INR currency, booking reference, event id and replay/idempotency.
OTP	Separate mobile and email challenge flows; expire codes; limit resend and attempts; store only secure verifier/hash where provider does not own verification.
Ticket delivery	Transactional outbox or durable queue for email and WhatsApp retries after confirmed payment.
QR validation	Authenticated staff endpoint; hash/verify credential; show-time window; atomic one-time redemption.

6. Non-functional requirements

Area	Requirement
Concurrency	Database-level arbitration prevents duplicate active seat allocations regardless of WebSocket message order.
Performance	Target 95th percentile under 500 ms for cached catalogue and under 1.5 s for seat map, excluding provider latency; validate against pilot load.
Availability	Target 99.5% monthly for the initial service, excluding scheduled maintenance and external provider outages.
Security	TLS; secure session/token lifecycle; passwordless OTP controls; least privilege; owner resource scoping; encrypted secrets; webhook signature verification; audit log.
Privacy	Collect only needed profile and ride data; consent and retention controls for notifications; redact sensitive data from logs.
Accessibility	Keyboard operable, labelled seat states, visible focus, reduced motion support and WCAG AA contrast target.
Localization	Jaipur-local date/time, INR, India country code and English-first copy; later add Hindi strings without changing API semantics.
Recovery	Database backups, payment reconciliation, expiry worker recovery and queued-notification retry/dead-letter handling.

7. State and error handling

- Seat: AVAILABLE → HELD → BOOKED, or HELD → AVAILABLE at expiry/release. Holds have a server-controlled expiry.
- Booking: PENDING_PAYMENT → CONFIRMED after verified payment; pending bookings may expire/cancel; refunds are tracked independently from cancellation.
- Ride request: REQUESTED → ACCEPTED or DECLINED; accepted membership consumes one available seat atomically.
- Retries with the same idempotency key return the prior result. A late payment after hold expiry follows a reconciliation/refund path and never silently reallocates an occupied seat.

8. Acceptance criteria

- Two clients selecting the same final seat cannot both receive a confirmed booking; the losing client receives an updated unavailable seat state.
- An expired unpaid hold becomes available again and a retrying client sees the refreshed state.
- A booking remains pending when a checkout redirect claims success but a valid webhook has not arrived.
- Only a verified owner of a theatre can edit its catalogue or shows.
- A confirmed booking creates one QR credential, can be delivered through configured channels, and cannot be redeemed twice.
- A ride request exposes participant chat only to the ride owner and accepted/requesting members according to the product policy.